Two Error Models

Document Number:	D4054R0
Date:	2026-03-12
Audience:	LEWG

Reply-to:	Vinnie Falco vinnie.falco@gmail.com

Table of Contents

Abstract

Revision History

- R0: March 2026 (post-Croydon mailing)

1. Disclosure

2. The Partition

- 2.1 Infrastructure Operations
- 2.2 Compound-Result Operations
- 2.3 Compound Results in Practice
- 2.4 Where the Partition Becomes a Problem

3. The Infrastructure Error Model

4. The Compound-Result Error Model

5. The Channel Split

- 5.1 Known Mappings
- 5.2 Prior Engagement

6. Four Defenses, Four Concessions

- 6.1 Accept the Information Loss
- 6.2 Route Everything Through `set_value`
- 6.3 Classify Errors Into Channels
- 6.4 Decompose with `let_value`
- 6.5 The Observable Tradeoffs

7. Domain Boundary

8. Conclusion

9. Acknowledgments

References

Abstract

Operations partition into two classes based on postcondition structure. Infrastructure operations have binary outcomes: the postcondition was satisfied or it was not. Compound-result operations produce a status and associated data, always paired. The sender three-channel model is correct for the first class. It cannot represent the second without losing data, bypassing the channels, or redefining what the channels mean.

This paper is one of a suite of five that examines the relationship between compound I/O results and the sender three-channel model. The companion papers are D4050R0[14], "On Task Type Diversity"; D4053R0[5], "Sender I/O: A Constructed Comparison"; D4055R0[12], "Consuming Senders from Coroutine-Native Code"; and D4056R0[13], "Producing Senders from Coroutine-Native Code."

Revision History

R0: March 2026 (post-Croydon mailing)

- Initial version.

---

1. Disclosure

The authors developed and maintain Corosio[3] and Capy[4] and believe coroutine-native I/O is the correct foundation for networking in C++. The authors provide information, ask nothing, and serve at the pleasure of the chair.

---

2. The Partition

2.1 Infrastructure Operations

`malloc`, `fopen`, `pthread_create`, GPU kernel launch, thread pool submit, timer arm. The operation either executes or it does not. An error means the postcondition was violated.

Operation	Success	Failure
`malloc`	Block returned	Allocation failed
`fopen`	File handle returned	Open failed
`pthread_create`	Thread running	Creation failed
GPU kernel launch	Kernel running	Launch failed
Thread pool submit	Task queued	Pool exhausted
Timer arm	Timer armed	Resource limit
Mutex acquire	Lock held	Deadlock / timeout

Every row is binary. The postcondition was satisfied or it was not.

2.2 Compound-Result Operations

Read, write, accept, parse, convert. The operation completes and returns a classification of what happened, paired with associated data.

Operation	Result
`read`	`(status, bytes_transferred)`
`write`	`(status, bytes_written)`
`from_chars`	`(ptr, errc)`
`strtol`	`(value, endptr, errno)`
`accept`	`(status, peer_socket)`

Two classes. The partition is not about asynchrony. It is about what the operation promises to return.

Some operations straddle the boundary. `connect` returns only a status code - no associated data. It is structurally a binary outcome: the connection was established or it was not. Windows `ConnectEx` accepts an optional send buffer, but the send byte count is available only through a separate `GetOverlappedResult` query - it is not part of the completion result. The completion itself delivers a status code. A bridge that recognizes status-only results can route `connect` through three channels without loss. `accept` cannot be reduced this way because the peer socket is associated data that would be destroyed on the error path.

`std::from_chars` is synchronous. `read` may be synchronous or asynchronous. Both return compound results where the status and the associated data are inseparable. The partition follows from postcondition structure, not from how the operation is dispatched.

The status code is not evidence of a postcondition violation. It is the postcondition. `read(fd, buf, n)` promises to report what happened on the descriptor. `from_chars(first, last, value)` promises to report how far it parsed and whether parsing succeeded. Every status code fulfills that promise:

error_code	bytes	What happened	Postcondition violated?
success	N	Full read	No
success	M < N	Partial read	No
`EOF`	0	Peer closed	No
`ECONNRESET`	0	Peer reset	No
`EWOULDBLOCK`	0	Try again	No
`EBADF`	0	Invalid fd	Yes

The distinguishing predicate is context-independence. `EBADF` is a postcondition violation in every protocol, every application, every call site - the file descriptor is invalid regardless of what the application intended to do with the data. `ECONNRESET` is fatal in an HTTP request handler but expected in a long-polling connection. `EOF` is an error when reading a fixed-length message but a normal termination signal when reading until close. A status code whose classification depends on protocol state is an outcome report, not a postcondition violation. Only `EBADF`-class errors - invalid file descriptor, bad address, not a socket - are postcondition violations. Every other outcome is the operation correctly reporting the state of the world.

2.3 Compound Results in Practice

The compound-result pattern is not a design preference. It is how systems report outcomes when the result carries more than a boolean. io_uring delivers `(res, flags)` in one CQE. IOCP delivers `(BOOL, lpNumberOfBytesTransferred, lpOverlapped)` in one call. POSIX `read()` returns `ssize_t` with

`errno`. `std::from_chars` returns `{ptr, ec}`. Three OS families and the C++ standard library converge on the same shape: status and data arrive as a pair because they are a single result.

2.4 Where the Partition Becomes a Problem

For synchronous code, compound results are straightforward. The function returns a struct or a tuple. The caller inspects both fields at the same call site. No routing decision is required.

`return` and `throw` are mutually exclusive paths. A `write` that throws on `ECONNRESET` destroys the 500-byte transfer count the application needed. The industry solved this decades ago: use the value channel whenever the result is not 100% failure. `std::from_chars` returns `{ptr, errc}`. POSIX `read()` returns a byte count. Asio returns `(error_code, size_t)`. The reason is information preservation, not performance.

The sender three-channel model presents the same challenge across three mutually exclusive paths instead of two.

---

3. The Infrastructure Error Model

The sender three-channel model assigns semantic meaning to each channel:

- `set_value(args...)` - the postcondition was satisfied; here are the results
- `set_error(err)` - the postcondition was violated; here is why
- `set_stopped()` - the operation was cancelled before attempting the postcondition

For infrastructure operations, this mapping is unambiguous:

```
// Timer sender: binary outcome
auto timer_sender = schedule_at(scheduler, deadline);
// Success: set_value()       - timer expired
// Failure: set_error(ec)     - resource exhaustion
// Cancel:  set_stopped()     - cancelled before expiry
```

```
// Thread pool submit: binary outcome
auto submit_sender = on(pool.get_scheduler(), work);
// Success: set_value(result) - work executed
// Failure: set_error(ec)     - pool rejected
// Cancel:  set_stopped()     - cancelled before dispatch
```

Channel assignment is deterministic. No information is lost. No adaptation is required. `then` sees the result. `upon_error` sees the error. `upon_stopped` sees the cancellation. The channels compose with generic algorithms: `retry` retries on `set_error`, `when_all` cancels siblings on failure, `upon_error` routes errors to handlers.

The three-channel model is correct for this class. `std::execution` serves it well.

---

4. The Compound-Result Error Model

The status code is not a failure signal. It is a vocabulary:

- `ECONNRESET` - the peer reset the connection
- `EPIPE` - the write end is closed
- `ETIMEDOUT` - the peer did not respond within the timeout
- `EWOULDBLOCK` - nothing available now; try again
- `EOF` - the peer closed the connection gracefully

Each requires different application handling. None indicates that the operation failed to operate. `ECONNRESET` means "fatal, abort the transaction" in an HTTP request handler, but "done, expected closure" in a long-polling connection that the server intentionally drops. The same error code is classified differently depending on the protocol state the application holds.

Partial results are normal. A `write` that sends 500 of 1,000 bytes before `ECONNRESET` produces `(ECONNRESET, 500)`. Both values are needed. The 500 tells the application how much of the message the API accepted for that operation. The status tells it why the rest was not.

Chris Kohlhoff identified this in P2430R0[2] ("Partial success scenarios with P2300," 2021):

> *"Due to the limitations of the `set_error` channel (which has a single 'error' argument) and `set_done` channel (which takes no arguments), partial results must be communicated down the `set_value` channel."*

POSIX has modeled I/O this way since 1988[9]. Asio since 2003. Go's `io.Reader` returns `(n int, err error)`. Rust's `Read::read` returns `Result<usize>` - byte count discarded on error. C# throws, same loss. The compound-result model is the older and more widespread convention. The question is not which convention is universal. It is whether the sender three-channel model can represent compound results without loss when the I/O layer delivers them.

5. The Channel Split

When a sender-based I/O operation completes, it must choose a channel. The byte count and the status code - which the OS delivered as a pair - are now split:

```
async_read(socket, buffer)
    | then([](size_t n) {
          // byte count visible
          // error_code invisible
      })
    | upon_error([](std::error_code ec) {
          // error_code visible
          // byte count invisible
      });
```

`then` and `upon_error` are mutually exclusive execution paths. The programmer cannot inspect both values at the same call site. The I/O result was a pair. The model split it.

To recover the pair, the programmer must defeat the channels:

```
async_read(socket, buffer)
    | into_expected
    | then([](std::expected<size_t, std::error_code> result) {
          // both visible again
      });
```

This works. But the result is now `set_value(expected<size_t, error_code>)`. The error code is inside the `expected`, invisible to `when_all`, `upon_error`, and `retry`. This is Approach A1 (D4053R0[5] Section 5) with the pair wrapped in `std::expected` instead of delivered as two arguments. The channel-based composition algebra is bypassed. `let_error` is the same pattern in reverse: catch the error, re-emit through `set_value`. Both recover the pair by routing everything through `set_value`.

The sender-native answer is `let_value`:

```
async_read(socket, buffer)
    | let_value(
          [](std::error_code ec, std::size_t n) {
              if (ec)
                  return just_error(ec);
              return just(n);
          });
```

This is Approach C in D4053R0[5] Section 8. The handler sees both values. Classification happens with full application context. Downstream, `upon_error` is reachable, `when_all` cancels siblings, `retry` fires.

Now write 1,000 bytes. The kernel accepts 500 before the connection dies. The handler sees `(ECONNRESET, 500)`. It emits `just_error(ECONNRESET)`. The 500 is gone. `set_error` takes a single argument. The decomposition point moved from the I/O layer to the application. The information loss did not.

5.1 Known Mappings

Two known attempts to solve the channel assignment problem for I/O illustrate the structural difficulty. Neither involves coroutines. Both operate at the receiver level: which of `set_value`, `set_error`, `set_stopped` does the I/O sender call when the kernel completion arrives?

The default mapping. beman.net[7], the reference sender-based networking implementation:

```
if (!ec)
    set_value(std::move(rcvr), n);
else
    set_error(std::move(rcvr), ec);
```

All non-zero error codes go to `set_error`. The byte count is discarded unconditionally. Total byte-count loss on any error.

The refined mapping. Peter Dimov proposed a convention in P4007R0[6] (Section 3.6) that preserves partial results by discriminating the channel based on the byte count and error code category:

Completion	Channel
`(eof, n)` for any n	`set_value(eof, n)`
`(canceled, n)` for any n	`set_stopped()`

Completion	Channel
`(ec, 0)` where ec is not eof	`set_error(ec)`
`(ec, n)` where n > 0	`set_value(ec, n)`

This is the only known mapping that preserves partial results on the error path. Dimov characterized it as "ad hoc."[6] The mapping requires semantic knowledge of specific error conditions: EOF is distinguished from cancellation, which is distinguished from all other errors. It is domain-specific I/O logic embedded in the channel routing.

Even this mapping has costs. Zero-byte errors lose data - they go to `set_error` with no byte count. The classification is context-free: the I/O layer decides which channel before the application can inspect the result. `ECONNRESET` is classified irrevocably at the I/O layer, before the protocol handler - the only code that knows whether `ECONNRESET` is fatal or expected - has a chance to see it.

Both mappings demonstrate the same structural problem: any function from `(error_code, size_t)` to `{set_value, set_error, set_stopped}` must either lose data, embed domain-specific classification at the wrong layer, or bypass the channels entirely. The problem is not the mapping. The problem is that the mapping is required at all.

5.2 Prior Engagement

Dietmar Kühl enumerated five channel-routing options for I/O in P2762R2[3] (Section 4.2), including fused `set_value(error_code, n)`, multiple `set_value` overloads, `set_error` routing, hybrid severity-based classification, and an `error_code` reference argument. Kühl acknowledged the partial-success problem directly: "some of the error cases may have been partial successes. In that case, using the set_error channel taking just one argument is somewhat limiting." He did not resolve it, noting that "when substantial work is done and partial successes become reasonable, it is likely that intermediate results are to be produced and algorithms of a different shape are used anyway." P2762R2 chose `set_error` routing (Approach B in D4053R0[5]) for its examples.

Kirk Shoop identified the same heuristic difficulty in P2471R1[10] ("NetTS, ASIO and Sender Library Design Comparison," 2021), observing that completion tokens translating to senders "must use a heuristic to type-match the first arg" and that the mapping creates "many different implementations of asSender."

P3570R2[4] ("Optional variants in sender/receiver," Fabio Fracassi, 2025) provides a mechanism for senders to present different completion types to coroutines than to direct receivers. The concrete use case is concurrent queues (P0260R19[11]): `set_error(conqueue_errc)` for receivers, `optional<T>` for coroutines. P3570

transforms ***which channel*** the coroutine sees, not the channel routing itself. It does not address compound I/O results: a sender completing with `set_error(ECONNRESET)` still discards the byte count regardless of how the coroutine receives the error.

To the author's knowledge, no published paper resolves the compound-result channel-routing problem identified in P2430R0[2]. The problem has been identified by Kohlhoff (2021), Kühl (2023), and Shoop (2021). It remains open.

---

6. Four Defenses, Four Concessions

Four positions are available to defend the three-channel model for I/O. Each concedes the thesis.

6.1 Accept the Information Loss

Argue that the byte count does not matter when an error occurs. This contradicts POSIX `write()`, Asio's twenty-year `(error_code, size_t)` convention, and network programming practice across every language that preserves compound results. Partial writes are normal.

6.2 Route Everything Through `set_value`

Call `set_value(error_code, bytes)` for all outcomes. The pair stays intact. But `set_error` and `set_stopped` now serve no purpose for I/O senders. `retry`, `when_all`, `upon_error` cannot distinguish success from failure. The three-channel model reduces to one channel with a structured result. The channels are not wrong. They are irrelevant.

6.3 Classify Errors Into Channels

Route "routine" errors through `set_value` and "exceptional" errors through `set_error`. This requires a taxonomy that does not exist in any OS API, redefines `set_value` to include errors, and forces the classification at the I/O layer - before the application, the only code with enough context to classify correctly, has seen the result. `ECONNRESET` is fatal in one protocol and expected in another. No context-free classification can get this right.

6.4 Decompose with `let_value`

The strongest sender-native response. Route everything through `set_value(error_code, bytes)`, pipe into `let_value`, inspect both values, re-route the error code to `set_error` (Section 5, Approach C in D4053R0[5] Section 8). The classification moves to application code with full protocol context.

But `set_error` takes a single argument. The byte count is destroyed when the error code crosses into the error channel. The decomposition point moved from the I/O layer to the application. The information loss did not.

6.5 The Observable Tradeoffs

Each position trades something the compound-result model preserves:

- (a) trades I/O data for channel simplicity
- (b) trades channel relevance for data preservation
- (c) trades fixed channel semantics for a domain-specific classification
- (d) trades error-path data for deferred, context-aware decomposition

The author is not aware of a fifth position and welcomes counterexamples (see D4053R0[5] Section 11).

---

7. Domain Boundary

The finding is not that the three-channel model is wrong. It is that the model has a domain. The partition exists independent of asynchrony. The channel problem exists because the sender model forces a routing decision.

Domain	Sync/Async	Error model	Three-channel model fits?
`malloc`	Sync	Infrastructure	Yes
`fopen`	Sync	Infrastructure	Yes
GPU dispatch	Async	Infrastructure	Yes
Thread pool	Async	Infrastructure	Yes
Timer	Async	Infrastructure	Yes
Connect	Either	Infrastructure	Yes
`from_chars`	Sync	Compound-result	No
`strtol`	Sync	Compound-result	No

Domain	Sync/Async	Error model	Three-channel model fits?
Read / Write	Either	Compound-result	No
Accept	Either	Compound-result	No
DNS resolve	Either	Compound-result	No

`accept` and DNS resolve straddle the boundary less cleanly than `read` / `write`. On failure, `accept` produces no socket - structurally closer to infrastructure. On success, both produce associated data (a peer socket, an address list) paired with a status. The compound-result classification follows from the success path: the data and the status are inseparable.

The reference implementation of `std::execution` (P2300R10[1]) - stdexec[8] - targets compile-time work graphs and GPU dispatch. Those are infrastructure operations with binary outcomes. The model is correct for its design domain.

The sender model forces exactly one layer to decompose the compound result. D4056R0[13] names this boundary the **abstraction floor**:

Region	What the code sees
Above the floor	`error_code` alone - composition works
Below the floor	`(error_code, size_t)` - both values intact

The coroutine-native model has no such boundary. Every layer sees both values.

---

8. Conclusion

Forty years of systems practice - POSIX, io_uring, IOCP, Asio, `from_chars` - chose information preservation. The three-channel model chose channel semantics. The trade-off is real.

---

9. Acknowledgments

The author thanks Chris Kohlhoff for identifying the partial-success problem in P2430R0[2], Dietmar Kühl for the channel-routing enumeration in P2762R2[3] and for `beman::execution`, Kirk Shoop for the completion-token heuristic analysis in P2471R1[10], Fabio Fracassi for P3570R2[4], Peter Dimov for the refined channel mapping,

Michał Dominiak, Eric Niebler, and Lewis Baker for `std::execution`, Maikel Nadolski for work on `execution::task`, Steve Gerbino for co-developing the constructed comparison, and Ville Voutilainen for reflector discussion on the partition.

References

1. P2300R10 - "std::execution" (Michał Dominiak et al., 2024). https://wg21.link/p2300r10
2. P2430R0 - "Partial success scenarios with P2300" (Chris Kohlhoff, 2021). https://wg21.link/p2430r0
3. P2762R2 - "Sender/Receiver Interface For Networking" (Dietmar Kühl, 2023). https://wg21.link/p2762r2
4. P3570R2 - "Optional variants in sender/receiver" (Fabio Fracassi, 2025). https://wg21.link/p3570r2
5. D4053R0 - "Sender I/O: A Constructed Comparison" (Vinnie Falco, Steve Gerbino, 2026). https://wg21.link/d4053r0
6. P4007R0 - "Senders and Coroutines" (Vinnie Falco, Mungo Gill, 2026). https://wg21.link/p4007r0
7. bemanproject/net - Sender/receiver networking library. https://github.com/bemanproject/net
8. stdexec - Reference implementation of std::execution. https://github.com/NVIDIA/stdexec
9. IEEE Std 1003.1-2024 - POSIX `read()` / `write()` specification. https://pubs.opengroup.org/onlinepubs/9799919799/
10. P2471R1 - "NetTS, ASIO and Sender Library Design Comparison" (Kirk Shoop, 2021). https://wg21.link/p2471r1
11. P0260R19 - "C++ Concurrent Queues" (Detlef Vollmann, Lawrence Crowl, Chris Mysen, Gor Nishanov, 2025). https://wg21.link/p0260r19
12. D4055R0 - "Consuming Senders from Coroutine-Native Code" (Vinnie Falco, Steve Gerbino, 2026). https://wg21.link/d4055r0
13. D4056R0 - "Producing Senders from Coroutine-Native Code" (Vinnie Falco, Steve Gerbino, 2026). https://wg21.link/d4056r0
14. D4050R0 - "On Task Type Diversity" (Vinnie Falco, 2026). https://wg21.link/d4050r0